

Unit 1, Assignment 3: Answer Key

C1. Without running the code, predict the output and justify your answer. (Tests understanding of dangling else.)

```
int a = 10, b = 20, c = 30;
if (a > b)
    if (b > c)
        printf("Case 1");
    else
        printf("Case 2");
else
    printf("Case 3");
```

Answer:

Output: Case 3 (verified by compiling and running this exact code)

Rule ("dangling else"): in C, an else always binds to the nearest preceding unmatched if. Here there are two if statements and two else clauses, so — despite the code looking ambiguous at a glance — each else does find a matching if: the first else binds to the inner if ($b > c$), and the second else binds to the outer if ($a > b$).

Since $a > b$ is $10 > 20$, which is false, the entire inner if/else block (testing $b > c$) is skipped completely — it is never reached at all — and control goes straight to the outer else, printing "Case 3". A common student mistake is to assume the innermost condition ($b > c$) is checked regardless; it is not, because the outer condition already failed.

C2. A student writes `float x = 1 / 3; printf("%f", x);` The output is 0.000000 instead of 0.333333. Identify two separate bugs/reasons causing this and correct the code.

Answer:

Reason 1 — Integer division: 1 and 3 are both written as int constants (no decimal point), so the expression $1 / 3$ is evaluated using integer division. Integer division discards the remainder, so $1 / 3$ evaluates to 0 (not 0.333...) before anything else happens.

Reason 2 — Declaring the destination as float does not fix it: this is the misconception that trips most students up. The type of `x` (float) has no effect on how $1 / 3$ is computed — the division is carried out first, using the types of the two operands (both int), giving the integer 0. Only after that is the already-truncated value 0 converted to 0.0 and stored in `x`. Declaring `x` as float only affects how the final result is stored, not how the division itself is performed.

Corrected code — at least one operand must be a floating-point value so that floating-point division is used:

```
float x = 1.0 / 3;
printf("%f", x); // prints 0.333333
```

($1.0 / 3$, $1 / 3.0$, and $(\text{float})1 / 3$ all work equally well — as soon as one operand is float/double, C performs the division as floating-point.)

C3. Write a single if condition (no nested if) to check whether a given year is a leap year (divisible by 4, but not by 100, unless also divisible by 400).

Answer:

```
#include <stdio.h>

int main() {
    int year;

    printf("Enter a year: ");
    scanf("%d", &year);

    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        printf("%d is a leap year\n", year);
    else
        printf("%d is not a leap year\n", year);

    return 0;
}
```

C4. Explain what happens if you use `=` instead of `==` inside an if condition. Give an example where this bug goes unnoticed by the compiler (no warning/error) but produces a wrong logical result.

Answer:

`=` is the assignment operator, not comparison. `if (a = 0)` does not test whether `a` equals 0 — it assigns 0 to `a`, and then the if condition evaluates the result of that assignment (which is 0). In C, any non-zero value in an if condition is treated as true and 0 is treated as false. So:

- `if (a = 0)` → always assigns 0 to `a`, and the condition is always false (since the assigned value is 0).
- `if (a = 5)` → always assigns 5 to `a`, and the condition is always true (since the assigned value is non-zero) — regardless of `a`'s previous value.

Either way, the variable's original value is silently overwritten, and the branch taken no longer depends on what the programmer intended to test.

Example where this compiles with no warning or error at all (verified: compiled successfully with plain "gcc file.c", producing zero warnings):

```
#include <stdio.h>
int main() {
    int a = 5;
    if (a = 0) {                // BUG: should be a == 0
        printf("true branch\n");
    } else {
        printf("false branch\n");
    }
    printf("a is now %d\n", a);  // a has been overwritten to 0!
    return 0;
}
```

Compiled with a plain "gcc file.c" (no extra flags), this produces no warning or error, yet `a`'s original value of 5 is silently destroyed and the wrong branch's logic runs. The compiler only warns about this ("suggest parentheses around assignment used as truth value") when extra warning flags such as `-Wall` are enabled — this was confirmed directly: the same file compiled clean with plain gcc, but produced a warning under gcc `-Wall`. This is exactly why it is good practice to always compile with `-Wall`, and why

some programmers write the constant first, e.g. if (0 == a), so that if == is mistyped as =, the compiler flags an error ("0 = a" is not assignable).

C5. Identify the logical flaw (not syntax error) in the code below and rewrite it correctly.

```
int marks = 85;
if (marks > 90)
    printf("Grade A");
if (marks > 80)
    printf("Grade B");
if (marks > 70)
    printf("Grade C");
```

Answer:

The logical flaw: these are three separate, independent if statements, not a chain — every single one of them is checked regardless of whether an earlier one already matched. For marks = 85, the first condition (marks > 90) is false and is skipped, but the second (marks > 80) is true, printing "Grade B", and the third (marks > 70) is also true, printing "Grade C" as well. So the actual output is "Grade BGrade C" (run together, since none of the printf calls contain a newline) — two grades are printed for a student who should receive only one.

Fix using an else-if ladder, checking from the highest cutoff down to the lowest, so that only the first matching branch executes and the rest are automatically skipped:

```
int marks = 85;
if (marks > 90)
    printf("Grade A");
else if (marks > 80)
    printf("Grade B");
else if (marks > 70)
    printf("Grade C");
else
    printf("Grade D or below");
```

Now marks = 85 correctly prints only "Grade B".

C6. A user enters 3.9 for an integer variable declared as int x; scanf("%d", &x);. What value does x actually store, and why? What happens to the leftover input, and how could this leftover affect a subsequent scanf("%f", &y) call?

Answer:

x stores 3. The %d conversion in scanf reads characters only as long as they form a valid integer. It reads the digit '3', and then stops the moment it encounters the '.' character, because a decimal point is not valid for an integer — it is not consumed.

What happens to the leftover input: the characters that were not consumed — here, ".9" — are left sitting in the input buffer (stdin). They are not discarded; they remain there waiting to be read by the next input operation.

Effect on a subsequent scanf("%f", &y): the very next scanf call does not wait for the user to type anything new. It immediately reads the leftover ".9" still sitting in the buffer, which is a perfectly valid representation of the float value 0.9, and stores y = 0.900000 — without ever giving the user a chance to enter a new value.

This was verified directly: feeding the single input "3.9" to a program that runs `scanf("%d", &x)` followed by `scanf("%f", &y)` produced $x = 3$ and $y = 0.900000$, exactly as explained above.

C7. Predict the output and explain the reasoning behind operator precedence and short-circuit evaluation. Why doesn't this cause a divide-by-zero runtime error?

```
int a = 0, b = 5, c;  
c = (a != 0) && (b / a > 1);  
printf("%d", c);
```

Answer:

Output: 0 (verified)

Explanation: `&&` is the logical AND operator, and C guarantees short-circuit evaluation for it — if the left-hand operand evaluates to false, the right-hand operand is not evaluated at all, because the overall result of a false AND anything is already known to be false.

Here, `(a != 0)` is `(0 != 0)`, which is false. Because the left operand is false, C never evaluates the right-hand operand `(b / a > 1)` at all — the division `b / a` (which would be a division by zero, undefined behavior / a runtime crash) simply never executes. The whole expression short-circuits to false (0), which is assigned to `c` and printed.

This is a very common and useful pattern in real code — placing a safety check (like `a != 0`, or a pointer non-NULL check) before an operation that would otherwise be unsafe, relying on short-circuit evaluation to skip the unsafe part when the check fails.

C8. Write a program that takes three boolean-like integers (0 or 1) as input representing switches, and turns a bulb ON only if exactly two switches are ON (not zero, one, or three). Use only if-else and logical operators (no loops).

Answer:

The truth table has exactly three winning combinations out of eight — the cases where exactly two of the three switches are 1: (1,1,0), (1,0,1), and (0,1,1). The condition below is built directly from those three combinations using logical operators (no counting/addition needed):

```
#include <stdio.h>  
  
int main() {  
    int s1, s2, s3;  
  
    printf("Enter three switch states (0 or 1): ");  
    scanf("%d %d %d", &s1, &s2, &s3);  
  
    if ((s1 && s2 && !s3) ||  
        (s1 && !s2 && s3) ||  
        (!s1 && s2 && s3)) {  
        printf("Bulb is ON\n");  
    } else {  
        printf("Bulb is OFF\n");  
    }  
  
    return 0;  
}
```

(An equally correct and often simpler alternative is if $(s1 + s2 + s3 == 2)$, which counts how many switches are 1. The explicit truth-table version above is shown because the question frames it as a truth-table exercise using logical operators.)

C9. A junior programmer claims char can never store a negative number. Prove or disprove this with a code example and explanation of how char is stored in memory (signed vs unsigned).

Answer:

This claim is false (disproved) in the common case. A char occupies 1 byte (8 bits) of memory. Whether plain char behaves as signed or unsigned is implementation-defined in the C standard — it depends on the compiler and platform — but on most common platforms and compilers (including GCC and Clang on x86/x64), plain char is signed by default, giving it a range of -128 to 127 using two's-complement representation, exactly like a signed integer type. In that (very common) case, char absolutely can store negative numbers.

However, if a variable is explicitly declared unsigned char, then it truly cannot store a negative number — all 8 bits are used for magnitude, giving a range of 0 to 255, and assigning a negative value causes it to wrap around.

Code demonstrating both cases (verified by compiling and running):

```
#include <stdio.h>

int main() {
    char c = -5;
    printf("%d\n", c);           // prints -5

    unsigned char u = -5;
    printf("%u\n", u);          // prints 251 (wraps around: 256 - 5)

    return 0;
}
```

Output confirmed by running the code: the signed char correctly prints -5, while the unsigned char wraps around and prints 251, since the bit pattern of -5 is reinterpreted as an unsigned value.

Correct, precise answer for an exam: "char can store negative numbers on most common implementations, where plain char is signed by default — but this is implementation-defined by the C standard, not guaranteed. Only unsigned char is guaranteed to never store a negative number."

C10. Without executing, determine the final value of x and explain each step. Discuss why this is undefined behavior in C and why different compilers may give different answers.

```
int x = 5;
x = x++ + ++x;
printf("%d", x);
```

Answer:

The honest, exam-correct answer is that this expression's result is undefined behavior (UB) in C, and there is no single "correct" final value that the C standard guarantees. The reason: x is modified more than once (once by x++, once by ++x) within the same expression, and there is no sequence point

between those two modifications that fixes their relative order or which "version" of x each part of the expression sees. The C standard explicitly says that modifying an object more than once between sequence points, with no defined ordering between those modifications, makes the behavior of the entire expression undefined — the compiler is free to compute it in any order, or even assume the situation never happens and optimize based on that assumption.

This was tested directly: compiling and running this exact code with GCC (at both -O0 and -O2) produced `x = 12` on this system — but per the standard, this exact value must not be treated as portable or guaranteed. A different compiler, a different version of the same compiler, or a different optimization level is permitted to produce a different value (commonly seen values in practice across various compilers include 12, 13, 15, or others).

What each piece would mean in isolation, if there were no conflict: `x++` (post-increment) uses the current value of x in the expression, then increments x afterward. `++x` (pre-increment) increments x first, then uses the new value. The conflict is that both operations are trying to read and update the same variable x as part of computing a single sum, with the standard placing no requirement on which happens "first".

The lesson for the exam: never write an expression that modifies the same variable more than once (via `++`, `--`, or assignment) without a sequence point separating those modifications. The safe, well-defined way to write equivalent logic is to split it into separate statements:

```
int x = 5;
int p = x++;    // p = 5, x becomes 6
int q = ++x;    // x becomes 7, q = 7
x = p + q;      // x = 12 (well-defined every time)
```

C11. The Fall-Through Trap. Predict the exact output, then explain why. Explain in your own words what "fall-through" means, why it happens here, and rewrite the code with the break statements needed to make it print only "Two".

```
int x = 2;
switch (x) {
    case 1: printf("One ");
    case 2: printf("Two ");
    case 3: printf("Three "); break;
    default: printf("Other ");
}
```

Answer:

Output: Two Three (with a trailing space, verified by compiling and running this exact code)

"Fall-through" means that once execution enters a matching case in a switch, it keeps executing every statement below it, one after another, straight through any following case or default labels — completely ignoring their labels — until it either hits a break statement or reaches the end of the switch block. A case label is only a landing point/entry marker for where execution jumps in; it is not a boundary that stops execution on its own.

Here, `x = 2` matches case 2, so execution jumps in there and prints "Two ". Since case 2 has no break, execution falls straight through into case 3 (even though x is not 3) and prints "Three " as well. Only then does it hit the break at the end of case 3 and stop, never reaching default. Most beginners wrongly assume the switch "chooses one case" like an else-if ladder and stops there automatically — but in C it does not, unless a break is explicitly placed.

Corrected code, so that x = 2 prints only "Two":

```
int x = 2;
switch (x) {
    case 1: printf("One "); break;
    case 2: printf("Two "); break;
    case 3: printf("Three "); break;
    default: printf("Other ");
}
```

C12. Why Can't switch Use Floats or Strings? Explain why the code fails to compile. Mention how switch is typically implemented internally, and why this makes floating-point or string case labels fundamentally incompatible with it — not just "against the rules", but why the rule exists. What data types are legal for switch expressions and case labels in C?

```
float x = 2.5;
switch (x) {
    case 2.5: printf("Two point five"); break;
    default: printf("Other");
}
```

Answer:

This was tested directly and confirmed to fail to compile, with GCC reporting: "switch quantity not an integer" and "case label does not reduce to an integer constant".

Why the rule exists (not just "the rules say so"): the C standard requires the switch expression to have integer type (after the usual integer promotions), and every case label to be a constant integer expression, evaluable at compile time. This is closely tied to how switch is typically implemented internally. When the case values are reasonably dense (close together, like 1, 2, 3, 4...), compilers commonly translate switch into a jump table — an array of addresses, one per possible integer value in range, where the switch value is used directly as an array index to jump straight to the matching code in constant time, with no comparisons at all. (When case values are sparse, compilers may instead generate a binary search or a chain of comparisons, but the constant-integer requirement on case labels still applies either way.)

Floating-point values are fundamentally incompatible with this: (1) floating-point numbers cannot reliably be used as array indices — 2.5 has no natural, exact integer position in a jump table. (2) Floating-point representation is inexact — a value that looks like 2.5 in source code, or a computed value intended to equal 2.5, may not have the exact same bit pattern as another 2.5, so exact equality comparison (which switch relies on) becomes unreliable, unlike with integers where exact equality is always well-defined. Strings are incompatible for a different reason: a string in C is really an array of characters (or a pointer to its first character) — comparing two strings for equality requires comparing their contents character by character (e.g. with strcmp), not comparing two scalar values directly the way switch compares its expression to each case label. switch was designed only to compare a single scalar (integer) value directly against other scalar constants — it has no built-in mechanism to walk through and compare an entire string.

Legal types for the switch expression and case labels in C: any integer type — int, char, short, long, long long, and their signed/unsigned variants — as well as enumeration (enum) constants, which are themselves integer values under the hood. float, double, arrays, strings, and structures are never legal.

C13. Dangling Else Strikes Again — Nested if Edition. Predict the output for marks = 60, then explain the bug. Explain exactly which if the else binds to, and rewrite the code using braces so that a mark of 60 correctly prints "Pass" while under 50 prints "Fail" and 75+ prints "Distinction".

```
if (marks >= 50)
    if (marks >= 75)
        printf("Distinction");
    else
        printf("Fail");
```

Answer:

For marks = 60, the output is: Fail (as stated in the question, and consistent with the dangling-else rule).

Explanation of the bug: by the dangling-else rule, an else always binds to the nearest preceding if that does not already have its own else — which is the inner if (marks >= 75), not the outer if (marks >= 50). So this code is actually interpreted as:

```
if (marks >= 50) {
    if (marks >= 75)
        printf("Distinction");
    else
        printf("Fail");
}
```

There is no else attached to the outer if at all — if marks < 50, absolutely nothing gets printed (a separate hidden bug). For marks = 60: the outer condition (60 >= 50) is true, so we enter the block; the inner condition (60 >= 75) is false, so its else runs, printing "Fail" — even though 60 is a perfectly reasonable passing mark. The programmer's intent (a three-way split: Fail / Pass / Distinction) was never actually written; only a two-way split (Distinction / Fail) exists, nested one level inside a check that silently does nothing when marks < 50.

Corrected code using explicit braces to create the intended three-way logic:

```
if (marks >= 50) {
    if (marks >= 75) {
        printf("Distinction");
    } else {
        printf("Pass");
    }
} else {
    printf("Fail");
}
```

Now: marks < 50 → "Fail"; 50 ≤ marks < 75 → "Pass"; marks ≥ 75 → "Distinction" — exactly as required.

C14. Combined Validation — Menu-Driven Calculator. Design a switch-based calculator (+ - * /) that also handles two edge cases: (1) division by zero, (2) an invalid menu choice/operator character. Write the full logic and explain, for each safeguard, what would go wrong if that specific check were removed.

Answer:

```
#include <stdio.h>
```

```

int main() {
    char op;
    double num1, num2, result;

    printf("Enter first number: ");
    scanf("%lf", &num1);
    printf("Enter operator (+, -, *, /): ");
    scanf(" %c", &op);
    printf("Enter second number: ");
    scanf("%lf", &num2);

    switch (op) {
        case '+':
            result = num1 + num2;
            printf("Result = %.2f\n", result);
            break;

        case '-':
            result = num1 - num2;
            printf("Result = %.2f\n", result);
            break;

        case '*':
            result = num1 * num2;
            printf("Result = %.2f\n", result);
            break;

        case '/':
            // Safeguard 1: division by zero
            if (num2 == 0) {
                printf("Error: Division by zero is not allowed\n");
            } else {
                result = num1 / num2;
                printf("Result = %.2f\n", result);
            }
            break;

        // Safeguard 2: invalid operator
        default:
            printf("Error: Invalid operator '%c'\n", op);
    }

    return 0;
}

```

Safeguard 1 — division-by-zero check (if (num2 == 0) ... else ...): if this check were removed and the program simply executed `result = num1 / num2;` directly, dividing by zero would not be caught or reported cleanly. For integer division, this triggers undefined behavior and typically crashes the program at runtime (commonly a "Floating point exception" / SIGFPE signal on Linux, terminating the program abruptly). For floating-point division (as used here with `double`), it does not crash, but silently produces `inf` (infinity) or `-nan` / `nan`, which then gets printed to the user as a nonsensical "Result" with no explanation — the program keeps running but gives a meaningless, confusing answer instead of a clear error message.

Safeguard 2 — default case for invalid operator: if the default case were removed (or simply left empty with no message), entering any character other than +, -, *, or / would cause the switch to match nothing and do absolutely nothing — the program would silently skip straight past the entire switch block, print no result and no error, and the user would be left confused about whether their input was accepted, ignored, or whether the program had a bug. Handling it explicitly in default gives the user clear, immediate feedback that their operator choice was not recognized.